Assignment 02 - Smart Drone Hangar

Cattolico Giuseppe, Muller Arthur

18 marzo 2026

1 Introduzione e Obiettivo

Il progetto "Smart Drone Hangar" ha l'obiettivo di realizzare un sistema embedded per la gestione automatizzata del decollo e dell'atterraggio di un drone. Il sistema è suddiviso in due macro-componenti interconnessi tramite linea seriale: un'unità di controllo locale basata su Arduino (Drone Hangar), incaricata di gestire i sensori e gli attuatori fisici, e un'unità remota su PC (Drone Remote Unit - DRU) dotata di interfaccia grafica (GUI) per l'invio dei comandi e il monitoraggio dello stato. L'intero firmware Arduino è stato progettato seguendo un'architettura basata su task concorrenti e Macchine a Stati Finiti (FSM) sincrone, garantendo reattività e una corretta temporizzazione delle operazioni.

2 Architettura Hardware

2.1 Schema Elettrico e Breadboard

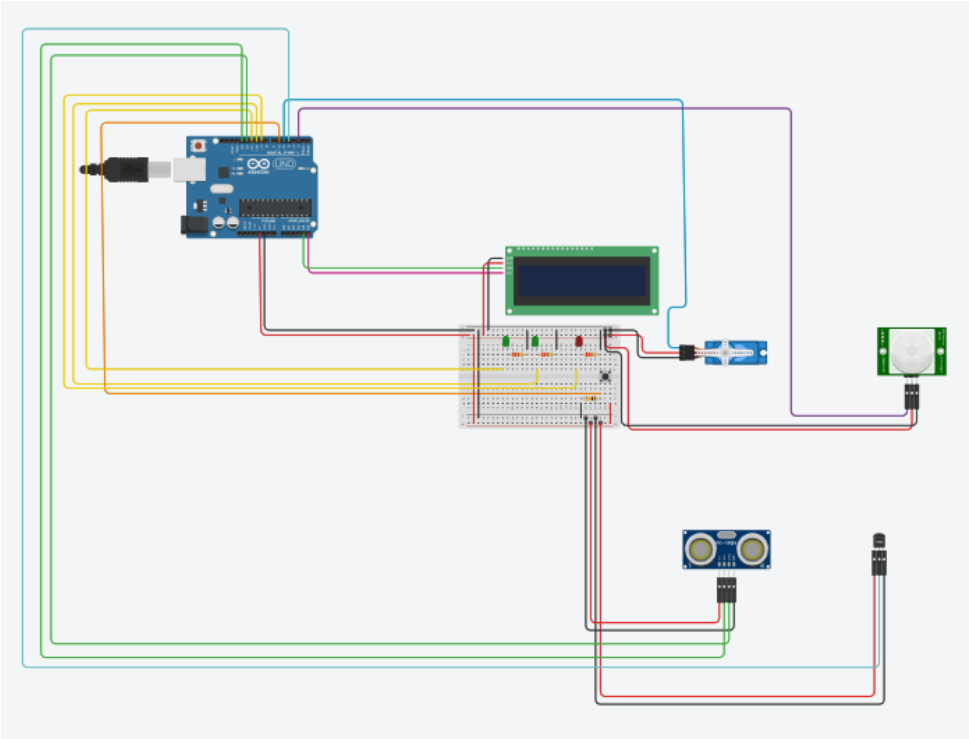

Figura 1: Rappresentazione della breadboard e schema circuitale.

2.2 Descrizione dei Componenti e Pin mapping

Il sottosistema locale si avvale dei seguenti componenti fisici per mappare l'ambiente e interagire con l'operatore. Di seguito è riportato il dettaglio dei collegamenti hardware al microcontrollore:

- **PIR Sensor (DPD) - `Pin 2`:** Rileva la presenza fisica del drone in prossimità della sommità dell'hangar durante le fasi di atterraggio.
- **Sensore Sonar (DDD) - `Trig: Pin 13, Echo: Pin 12`:** Misura costantemente la distanza del drone quando si trova all'interno o in prossimità della base, permettendo di validare l'effettivo completamento del decollo o dell'atterraggio.
- **Servo Motore (HD) - `Pin 5`:** Attuatore che controlla l'apertura e la chiusura della porta dell'hangar.
- **Display LCD I2C (16x2) - `Indirizzo 0x27`:** Fornisce un feedback visivo immediato all'operatore locale sullo stato del sistema (es. *DRONE INSIDE*, *TAKE OFF*, *ALARM*).
- **Sensore di Temperatura (TEMP) - `Pin 4`:** Monitora le condizioni termiche interne per prevenire surriscaldamenti durante le fasi operative.
- **LED di Segnalazione - `L1: Pin 9, L2: Pin 10, L3: Pin 11`:** L1 (verde) indica l'operatività del sistema, L2 (verde) lampeggia con periodo di 0.5s durante le manovre di decollo/atterraggio, e L3 (rosso) segnala uno stato di allarme critico.
- **Pulsante Tactile (RESET) - `Pin 6`:** Permette all'operatore di ripristinare il sistema in seguito alla risoluzione di un'anomalia termica.

3 Architettura Software

3.1 Struttura del Sistema

Il firmware è strutturato in task cooperativi eseguiti da uno scheduler personalizzato. Questa scelta architetturale permette di gestire in modo non bloccante la lettura dei sensori (Sonar, PIR, Temperatura), il controllo del servomotore, l'aggiornamento dell'interfaccia LCD e la comunicazione bidirezionale tramite porta seriale con l'applicazione PC. Il comportamento logico del sistema è governato da macchine a stati finiti che valutano le transizioni in base agli input dei sensori, ai comandi ricevuti dal DRU e a specifici timeout temporali (T1, T2, T3, T4).

3.2 Diagrammi delle Macchine a Stati (FSM)

3.3 I 4 Task Concorrenti e il loro Funzionamento

Il sistema è stato suddiviso in quattro task indipendenti, ognuno con una precisa responsabilità operativa (Separation of Concerns), descritti di seguito:

- **MainHangarTask (Logica di Movimento):** Gestisce il comportamento primario del hangar. Reagisce ai comandi (`takeOffCommand`, `landingCommand`) a condizione che non vi siano allarmi attivi. Attende le conferme spaziali e temporali dai sensori (Sonar e PIR) per completare le transizioni di stato. Si occupa inoltre di pilotare il servomotore della porta e aggiornare il display LCD.

- **AlarmTask (Monitoraggio Sicurezza):** Valuta costantemente la temperatura in modo slegato dalle operazioni di volo. In caso di innalzamento termico, porta il sistema in *Pre-alarm* (inibendo nuove manovre) o in *Alarm* (bloccando il `MainHangarTask`, chiudendo la porta e accendendo il LED rosso L3 fino alla pressione del tasto di reset).

- **BlinkingLedTask (Feedback Visivo):** Fa lampeggiare il LED verde L2 durante le fasi transitorie. Controlla continuamente se il sistema è in fase di decollo o atterraggio e, in caso affermativo, alterna lo stato del LED ogni 250ms, generando un periodo complessivo di 500ms.

- **SerialTask (I/O e Comunicazione):** Gestisce la comunicazione con l'unità remota (DRU). Ascolta i comandi in arrivo (es. `TAKEOFF`, `LAND`) e invia periodicamente (ogni 500ms) una stringa di riepilogo con lo stato aggiornato dell'hangar.

3.4 Vantaggi dell'Architettura a Task

L'adozione di una soluzione basata su 4 task cooperativi garantisce tre vantaggi fondamentali rispetto a un tradizionale approccio monolitico:

1. **Esecuzione non bloccante:** Evitando l'uso di funzioni bloccanti come `delay()`, lo scheduler può eseguire operazioni periodiche (come il lampeggio del LED) senza impedire al microcontrollore di continuare a leggere sensori o ascoltare la seriale.

2. **Affidabilità della Sicurezza:** L'`AlarmTask` viene eseguito in parallelo. Ciò assicura che il controllo termico non venga mai interrotto o ritardato dalle tempistiche meccaniche della porta o da eventuali latenze di rete.

3. **Modularità:** La netta separazione delle logiche rende il firmware facilmente manutenibile ed espandibile.

3.5 Gestione dello Stato Condiviso: Il Context

Per far comunicare i task senza accoppiarli rigidamente, è stato implementato il pattern architetturale *Shared State Object* tramite la classe `Context`. I task non comunicano mai direttamente tra loro. Il `Context` funge da "bacheca condivisa":

- **Scrittura:** Il `SerialTask` riceve un comando e invoca, ad esempio, `setTakeOffCommand()`, demandando l'esecuzione effettiva.

- **Lettura:** Il `MainHangarTask` interroga ciclicamente il `Context` (`isTakeOffCommanded()`) per avviare le proprie routine, mentre il `BlinkingLedTask` legge flag come `isTakingOff()` per avviare il lampeggio.

Questo approccio nasconde l'implementazione delle singole variabili e mantiene il sistema coerente e disaccoppiato.

3.6 Comunicazione Seriale asincrona: MsgService

La comunicazione con il PC avviene in modo asincrono e non bloccante grazie alla classe `MsgService`. Il meccanismo sfrutta la funzione di sistema `serialEvent()`, la quale viene chiamata automaticamente alla fine di ogni ciclo `loop()` se vi sono byte nel buffer hardware:

- **Ricezione in background:** I caratteri in arrivo vengono accumulati in un buffer testuale. Quando viene rilevato il carattere terminatore di a capo (`\n`), la stringa viene impacchettata in un oggetto `Msg` e viene alzato il flag `msgAvailable`.
- **Elaborazione:** Il `SerialTask`, durante il suo normale ciclo, verifica la disponibilità del messaggio tramite `isMsgAvailable()` ed eventualmente lo processa, senza mai doversi fermare in attesa sulla seriale.
- **Invio:** La funzione `sendMsg()` effettua un wrap della chiamata nativa `Serial.println()`, formattando correttamente i dati in uscita per la GUI remota.

4 Note Operative

4.1 Note Operative e Parametrizzazione

Il prototipo è stato sviluppato tenendo in forte considerazione la flessibilità in fase di test e calibrazione. Tutti i parametri fisici e temporali richiesti dalle specifiche (*D1, D2, T1, T2, T3, T4, Temp1, Temp2*) sono stati implementati come costanti (`#define`) o variabili di configurazione facilmente modificabili.

5 Difficoltà Riscontrate e Conclusioni

5.1 Difficoltà Riscontrate

Durante la fase di sviluppo e implementazione del prototipo, abbiamo affrontato e risolto diverse sfide tecniche, sia hardware che software:

- **Lettura dei sensori e librerie esterne:** Abbiamo optato per l'utilizzo di un sensore di temperatura DHT11. Tuttavia, l'integrazione di questo componente e delle relative librerie di terze parti ha causato difficoltà nella lettura dei dati, introducendo latenze e conflitti con la natura della nostra architettura a task.
- **Gestione dello stato condiviso:** Nelle prime iterazioni del codice, la classe `Context` non era strutturata in modo del tutto completo. Questo ha generato alcune difficoltà nel corretto aggiornamento e nella sincronizzazione delle variabili di stato, portando a comportamenti imprevisti. La revisione e il completamento del pattern *Shared State* hanno poi risolto definitivamente questi conflitti tra task.
- **Letture anomale del Sonar (Overflow):** Abbiamo riscontrato un'anomalia fisica legata al sensore a ultrasuoni. Quando la porta dell'hangar era aperta e non vi erano ostacoli vicini, il Sonar misurava una distanza fuori scala, restituendo un valore negativo che corrompeva la logica di controllo (simulando falsamente l'avvenuto atterraggio/decollo). Come visibile nel video allegato, abbiamo risolto il problema

in modo pratico posizionando un oggetto dietro la porta aperta, garantendo così una lettura di "background" fissa e sicura per il sensore.

- **Comunicazione Seriale PC-Arduino:** L'implementazione dell'interfaccia di comunicazione tra l'applicazione Java (GUI remota) e la porta seriale di Arduino ha richiesto particolare attenzione. Si sono verificati problemi iniziali di sincronizzazione e perdita di pacchetti tra i due ambienti.

5.2 Conclusioni

Lo sviluppo dello *Smart Drone Hangar* ha confermato la netta superiorità di un'architettura software basata su task concorrenti e Macchine a Stati Finiti (FSM) rispetto a un approccio procedurale monolitico.

Nonostante le difficoltà iniziali legate all'integrazione dei sensori e alla comunicazione seriale, la separazione delle responsabilità (*Separation of Concerns*) tra logica di movimentazione, sicurezza termica e I/O ha permesso di isolare e risolvere i bug in modo mirato.

In conclusione, il sistema finale risulta reattivo e deterministico: i task critici, come l'`AlarmTask`, intervengono tempestivamente senza mai essere ritardati o interrotti dalle operazioni meccaniche del servomotore o da eventuali latenze della GUI remota.

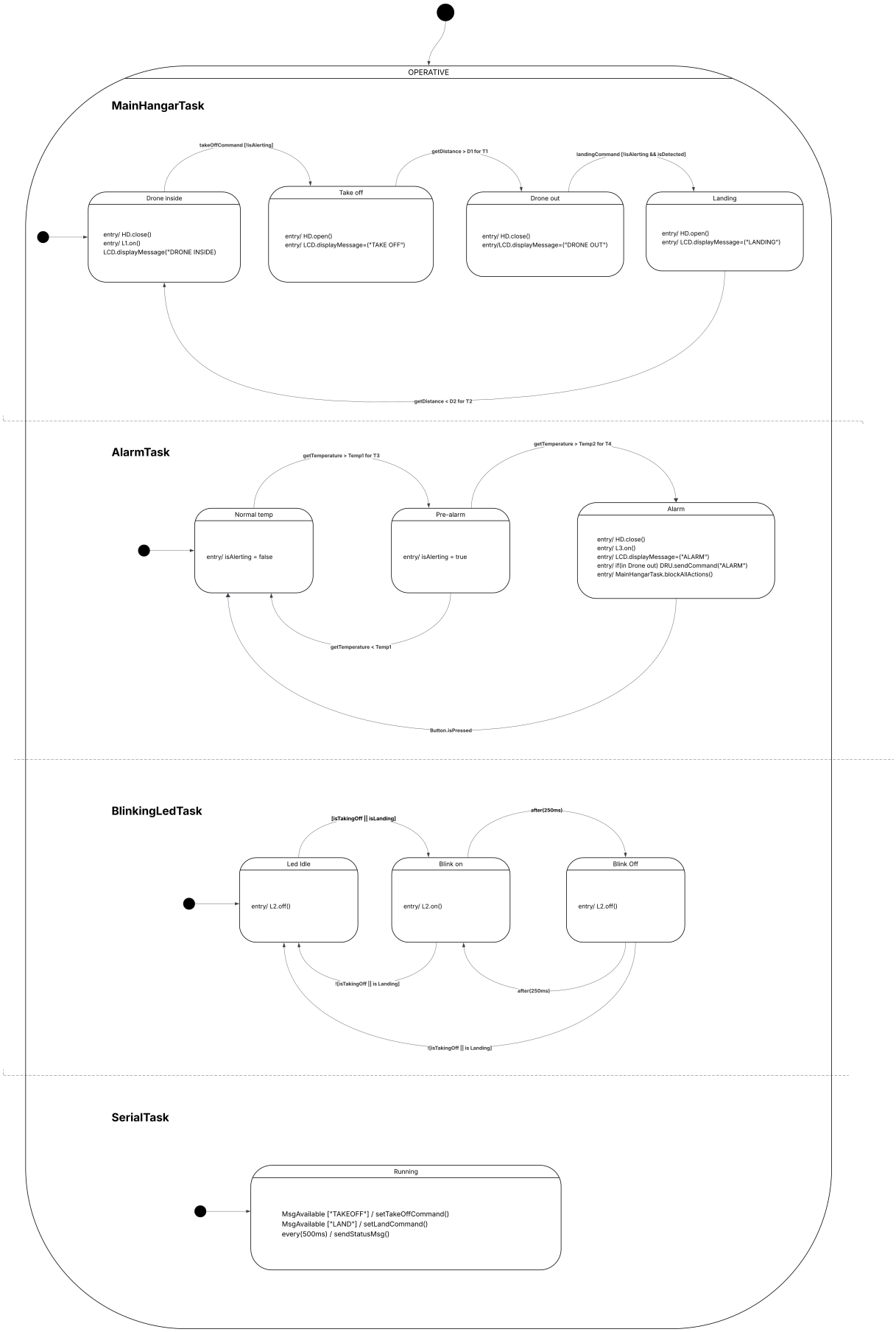

Figura 2: Diagramma della Macchina a Stati Finiti del sistema.